Progetto - Piattaforma Serverless per lo Screening e Parsing Automatico dei CV

Università degli Studi di Napoli Federico II
Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione DIETI
Laurea Magistrale in Ingegneria Informatica

Esame di Cloud Platforms and Infrastructure-as-Code, Prof. Canonico Roberto
Studente: Luigi Caretti M63001721

1. Introduzione e Requisiti del Sistema
    1.1 Requisiti Funzionali
    1.2 Requisiti Non Funzionali
2. Architettura del Sistema e Pattern Progettuali
    2.1 Pattern Architetturali Adottati
        2.1.1 Event-Driven Architecture (EDA)
        2.1.2 Direct Upload via Presigned URL
        2.1.3 Two-Bucket Pattern (Landing vs. Archive)
3. Fondamenti Teorici
    3.1 State Management con Remote Backend (S3)
        Configurazione del Backend HCL
    3.2 Disaccoppiamento e Resilienza della Messaggistica
        3.2.1 Amazon SQS: Modello a Code, Visibility Timeout e Long Polling
        3.2.2 Poison Messages e Redrive Policy (Dead Letter Queue)
    3.3 Computazione Serverless e Modelli di Sicurezza IAM
        3.3.1 AWS Lambda: Ciclo di Vita ed Ambiente di Esecuzione
        3.3.2 Sicurezza IAM
            Trust Policy
        Identity-Based Access Policy
    3.4 Persistence Layer e Modellazione NoSQL
        3.4.1 Amazon DynamoDB: Architettura Schemaless e Modalità Pay-Per-Request
        3.4.2 Modellazione degli Access Pattern e Global Secondary Index (GSI)
            Struttura del GSI
    3.5 Integrazione Event-Driven e Resource-Based Policies
        S3 Bucket Notification
        SQS Queue Policy e Convalida della Sorgente
4. Implementazione dell'Infrastruttura
    4.1 Organizzazione dei Moduli
    Implementazioni Future
        Monitoring e Alerting

1. Introduzione e Requisiti del Sistema

Nel settore delle risorse umane (Human Resources), la fase di *screening* preliminare dei Curriculum Vitae (CV) rappresenta uno dei principali colli di bottiglia operativi. La gestione manuale di elevati volumi di candidature comporta tempistiche dilatate, suscettibilità ad errori umani e difficoltà nel confrontare in modo oggettivo le competenze dei candidati rispetto alla posizione lavorativa aperta.

L'avvento dei modelli di linguaggio di grande scala (**Large Language Models - LLM**) offre l'opportunità di automatizzare l'estrazione di informazioni non strutturate (come i documenti PDF caricati dai candidati) trasformandole in strutture dati semantiche e tipizzate (formato JSON).

L'obiettivo principale del progetto è la progettazione, l'implementazione e il deployment di una pipeline serverless event-driven e cloud-native su Amazon Web Services (AWS), definita ed erogata interamente tramite **Infrastructure as Code (IaC)** con **Terraform**. Il sistema consente:

1. L'ingestione sicura dei file da parte dei client.
2. Il disaccoppiamento del flusso di elaborazione tramite code di messaggi.
3. Il parsing e la valutazione automatica dell'affinità del candidato mediante modelli generativi erogati da AWS Lambda.
4. La persistenza indicizzata su database NoSQL per consentire ricerche ad elevate prestazioni da parte dei recruiter.

1.1 Requisiti Funzionali

I requisiti funzionali (RF) definiscono le capacità operative e i flussi di elaborazione forniti dal sistema:

- **RF-01 (Caricamento Sicuro del Documento tramite API):** Il sistema deve consentire al client l'upload del CV convertendo il file in formato Base64 e inviandolo come payload JSON tramite una richiesta HTTP POST ad un endpoint Amazon API Gateway. Una Lambda API di backend si occuperà di decodificare il payload e salvare l'oggetto binario all'interno del bucket di staging Amazon S3 (cv-incoming-bucket), garantendo la corretta autenticazione IAM.
- **RF-02 (Notifica ed Inoltro Asincrono):** Il caricamento di un nuovo documento su S3 deve scatenare automaticamente un evento (`s3:ObjectCreated`) indirizzato ad una coda di messaggi Amazon SQS (`cv-processing-queue`) per disaccoppiare l'ingestione dall'elaborazione.
- **RF-03 (Parsing ed Estrazione dei Dati via LLM):** Una funzione AWS Lambda deve consumare i messaggi presenti nella coda SQS, prelevare il file dal bucket di staging, invocare l'LLM per estrarre le informazioni chiave (anagrafica, competenze, esperienze lavorative) e calcolare uno *score* numerico di adeguatezza.
- **RF-04 (Archiviazione e Segregazione del File):** A completamento del parsing, la funzione Lambda deve archiviare il CV originale all'interno di un bucket protetto di conservazione a lungo termine (`cv-archive-bucket`) e rimuovere contestualmente l'oggetto temporaneo dal bucket d'ingresso.
- **RF-05 (Persistenza dei Metadati):** Il JSON strutturato generato dall'LLM deve essere salvato su una tabella Amazon DynamoDB (`CandidatesTable`).
- **RF-06 (Consultazione, Ranking e Visualizzazione per Recruiter):** Il sistema deve garantire la ricerca dei candidati filtrata per mansione lavorativa (job_id) e ordinata in modo decrescente per punteggio (score), mediante l'interrogazione di un indice secondario globale (GSI). In fase di lettura, l'API Lambda deve generare una Presigned URL temporanea associata al bucket di archivio per permettere al Recruiter la visualizzazione sicura del PDF direttamente dal browser.

1.2 Requisiti Non Funzionali

I requisiti non funzionali (RNF) stabiliscono i vincoli di qualità, prestazionali e di sicurezza dell'infrastruttura:

- **RNF-01 (Scalabilità ed Event-Driven Computing):** L'architettura deve fondarsi interamente su servizi gestiti *On-Demand* e *Pay-Per-Request*. Il sistema deve essere in grado di scalare automaticamente a zero risorse quando inutilizzato e di tollerare picchi improvvisi di upload senza provisioning di istanze fisse.
- **RNF-02 (Resilienza e Tolleranza ai Guasti):** La pipeline deve garantire che nessun messaggio vada perso in caso di malfunzionamenti temporanei dell'LLM o dei servizi downstream, integrando politiche di *retry*, *visibility timeout* adeguati (1080s) e isolamento dei messaggi dannosi via DLQ.

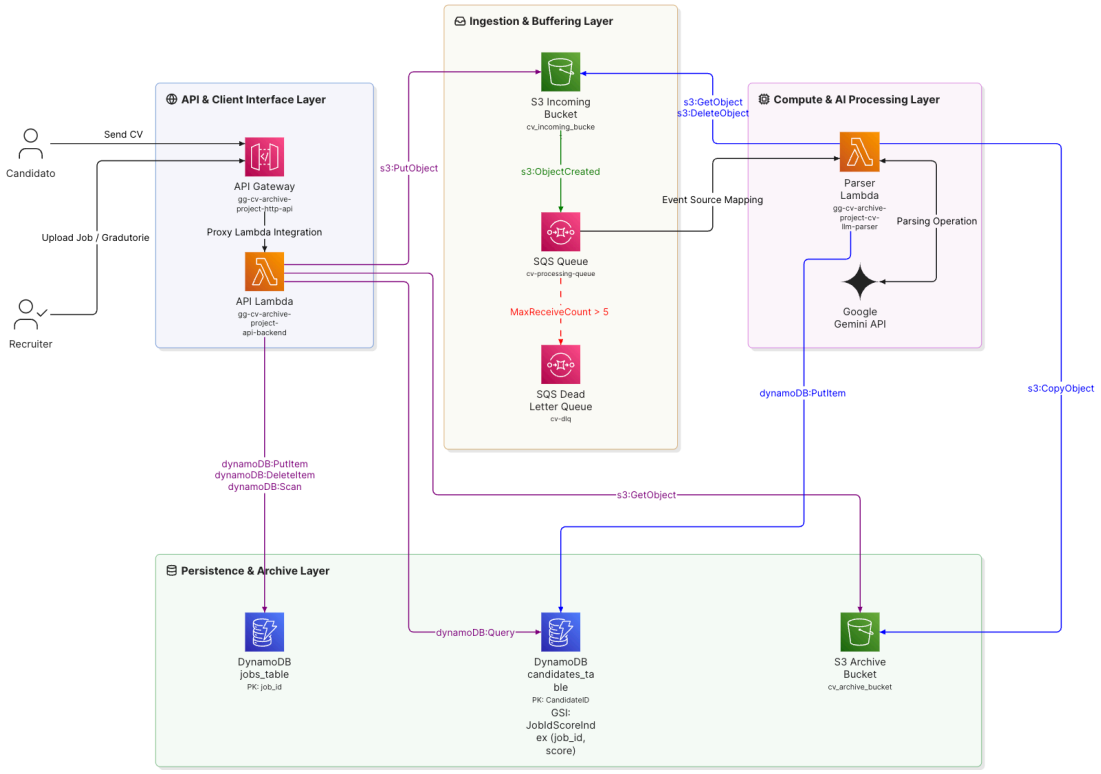
- **RNF-03 (Sicurezza e Principio del Minimo Privilegio):** Ogni componente deve operare nel rispetto del principio del minimo privilegio (Least Privilege Access). Le funzioni di calcolo (AWS Lambda) devono accedere alle risorse unicamente mediante Ruoli IAM con permessi granulari, mentre l'accesso al bucket d'ingresso deve essere ristretto dalle politiche CORS.
- **RNF-04 (Mantenibilità e Riproducibilità tramite IaC):** L'intera infrastruttura cloud deve essere dichiarata, tracciata e riproducibile tramite script Terraform versionabili. Lo stato dell'infrastruttura (`terraform.tfstate`) deve essere memorizzato in modo remoto e protetto da concorrenza.
- **RNF-05 (Efficienza dei Costi):** L'allocazione delle risorse (RAM della Lambda a 256 MB, timeout a 180s, storage S3 temporaneo) deve rientrare nei limiti di sostenibilità economica del Free Tier di AWS o minimizzare l'impatto economico a consumo.

2. Architettura del Sistema e Pattern Progettuali

L'architettura proposta implementa una pipeline cloud-native disaccoppiata e completamente gestita su Amazon Web Services (AWS), concepita per l'elaborazione ad alte prestazioni e a basso costo di documenti non strutturati. Il sistema si articola su due flussi concettuali distinti: il **Flusso di Ingestione ed Elaborazione (Write Path)** e il **Flusso di Consultazione (Read Path)**.

ATS Cloud Platform Architecture

Serverless AI pipeline: CV upload, SQS buffering, Gemini scoring and DynamoDB/S3 persistence



Per completezza forniamo in seguito una lista numerata che rappresenti il ***Data Flow della pipeline***:

1. **Ingestion Layer:** Il file del CV viene caricato nel bucket S3 di input (`cv-incoming-bucket`).
2. **Notification & Queuing Layer:** L'evento `s3:ObjectCreated` invia un messaggio con i dettagli del file caricato alla coda SQS (`cv-processing-queue`).
   a. Se l'elaborazione fallisce per 3 volte, SQS sposta il messaggio nella `cv-dlq`.
3. **Compute Layer:** La funzione AWS Lambda preleva i messaggi dalla coda SQS, scarica il contenuto del CV da S3 e invia il testo all'LLM tramite un prompt di astrazione.

   L'LLM restituirà un JSON struttura contenente le seguenti informazioni:

- Dati Anagrafici e ruolo desiderato.
- Anni di esperienza lavorativa.
- Array delle competenze tecniche.
- Score di affinità (0-100) calcolato in base alle competenze richieste.

4. **Persistence Layer (Dual-Storage Pattern):**
   - **DynamoDB (`CandidatesTable`):** Archivia le informazioni strutturate (metadati, competenze, score, stato) con `candidate_id` come Partition Key, abilitando ricerche e filtri per il team HR.
   - **S3 (`cv-archive-bucket`):** Archivia il documento originale per successive consultazioni umane, mentre il file temporaneo nel bucket di incoming viene eliminato.

2.1 Pattern Architetturali Adottati

2.1.1 Event-Driven Architecture (EDA)

L'architettura ad eventi rappresenta il paradigma fondamentale per garantire il disaccoppiamento temporale e logico tra i componenti del sistema.

In un'architettura sincrona tradizionale, il caricamento del file e la successiva elaborazione avverrebbero all'interno di una singola transazione HTTP, esponendo il sistema a colli di bottiglia, potenziale esaurimento dei thread e fallimenti a catena in caso di latenza da parte dell'LLM.

L'adozione del pattern Event-Driven tramite il middleware di messaggistica Amazon SQS offre i seguenti vantaggi:

- **Smoothing del Traffico (Spike Arresting):** In presenza di caricamenti massivi ed improvvisi di CV, la coda SQS funge da buffer temporaneo, assorbendo il carico e consentendo alla funzione Lambda di consumare i messaggi al proprio ritmo di esecuzione.
- **Isolamento dei Guasti:** Il fallimento dell'elaborazione di un singolo documento non impatta la disponibilità complessiva del sistema né l'ingestione di nuovi CV.

2.1.2 Direct Upload via Presigned URL

Per ottimizzare le prestazioni di rete ed eliminare costi infrastrutturali inutili, l'architettura evita il passaggio dei dati attraverso server applicativi intermedi o API Gateway.

Il pattern *Presigned URL* prevede che l'applicazione generi una firma crittografica temporanea contenente le credenziali IAM necessarie per eseguire l'operazione `s3:PutObject`. Il client utilizza questo URL pre-firmato per stabilire una connessione diretta HTTP `PUT` verso l'endpoint di Amazon S3.

2.1.3 Two-Bucket Pattern (Landing vs. Archive)

La scelta di adottare due bucket fisicamente separati (`cv-incoming-bucket` e `cv-archive-bucket`) risponde a rigorose esigenze di progettazione cloud-native:

1. **Prevenzione dei Cicli Ricorsivi di Eventi:** Se l'ingestione e la conservazione risiedessero nello stesso bucket, la scrittura del file finale da parte della Lambda genererebbe un nuovo evento `s3:ObjectCreated`. In assenza di filtri complessi, questo scatenerebbe un loop infinito di notifiche verso SQS e invocazioni della Lambda, con conseguente saturazione dei costi. La separazione fisica elimina il rischio alla radice.
2. **Segregazione degli Accessi (Least Privilege):** Il bucket d'ingresso deve essere esposto all'esterno (tramite politiche CORS e Presigned URL) per consentire gli upload dei client. Il bucket di archiviazione è invece totalmente privato, privo di configurazioni CORS e accessibile esclusivamente dall'IAM Role della funzione Lambda.

3. Fondamenti Teorici

In questo capitolo si è voluto introdurre alcuni dei fondamenti teorici necessari per comprendere a pieno il lavoro svolto sul codice.

3.1 State Management con Remote Backend (S3)

Terraform mantiene una mappatura esplicita tra le risorse dichiarate nel codice e le risorse reali create sul cloud provider all'interno di un file di stato denominato `terraform.tfstate`.

Nelle architetture di livello enterprise, la conservazione dello stato in locale è considerata un anti-pattern a causa di rischi legati alla perdita dei dati, alla mancata tracciabilità delle modifiche e all'impossibilità di collaborare in modo sicuro.

Nello scenario di questo progetto, la conservazione dello stato in locale è stata superata mediante la configurazione di un **Remote Backend su Amazon S3**. Questa scelta architetturale soddisfa i requisiti di persistenza, tracciabilità e sicurezza abilitando anche il meccanismo di **Native State Locking.** Il quale permette l'accesso esclusivo in scrittura al file in un ambiente multi-developer.

Configurazione del Backend HCL

`provider.tf`

```
# Configuration of the S3 backend for Terraform remote state. (Secure)
    backend "s3" {
        bucket = "gg-cv-test-bucket-state-00000" # Unique identifier of the S3 bucket wher
e the Terraform state file will be stored.
        key    = "dev/terraform.tfstate" # Path of the state file within the S3 bucket.
        region = "eu-north-1" # Hardcoded because the S3 resource has already been created
previously.
        encrypt = true # Enables encryption of the state file.
        use_lockfile = true # Enables locking of the state file.
    }
```

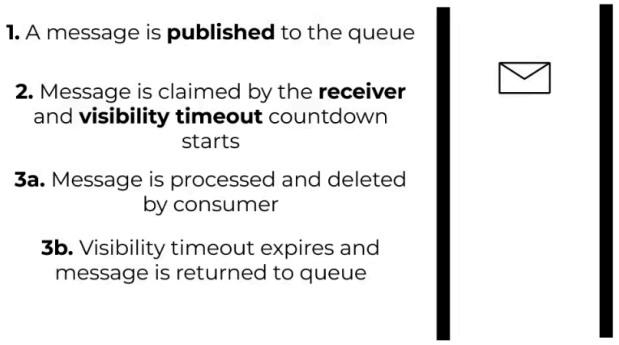
3.2 Disaccoppiamento e Resilienza della Messaggistica

3.2.1 Amazon SQS: Modello a Code, Visibility Timeout e Long Polling

Amazon SQS (Simple Queue Service) è un servizio di accodamento messaggi fully managed che consente il disaccoppiamento tra componenti distribuiti. Nella pipeline in esame viene utilizzata una **SQS Standard Queue**, che offre throughput illimitato di transazioni al secondo e garanzia di consegna *at-least-once*.

I due parametri operativi fondamentali per il corretto funzionamento del disaccoppiamento sono:

- **Visibility Timeout:** Quando il servizio Event Source Mapping di AWS Lambda preleva un messaggio dalla coda, SQS non lo elimina immediatamente, ma lo imposta in uno stato tale da 'oscurarlo' in modo per impedire che altre istanze della Lambda elaborino lo stesso file. Se la Lambda completa l'elaborazione con successo, invia una chiamata `sqs:DeleteMessage`. Se la Lambda fallisce o va in timeout, il messaggio torna visibile nella coda allo scadere del tempo prefissato. Il parametro è stato impostato a 1080s (18 minuti), rispettando la regola di sicurezza:
$$\text{SQS Visibility Timeout} \geq 6 \times \text{Lambda Timeout} \quad (6 \times 180\text{s} = 1080\text{s})$$
- **Long Polling (`ReceiveWaitTimeSeconds`):** Le chiamate API d'ispezione attendono l'arrivo effettivo di un messaggio prima di ritornare, riducendo drasticamente il numero di richieste API effettuate e azzerando i costi di rete a coda vuota.



3.2.2 Poison Messages e Redrive Policy (Dead Letter Queue)

Un *poison message* è un payload che, a causa di formattazioni corrotte, eccezioni non gestite nel codice o bug nell'LLM, provoca il fallimento sistematico della funzione di elaborazione ogni volta che viene letto.

Senza un meccanismo di isolamento, un messaggio malformato verrebbe rielaborato all'infinito, saturando le risorse di calcolo e bloccando la coda.

Per garantire la resilienza, è stata implementata una **Dead Letter Queue (DLQ)** governata da una `redrive_policy`. Il parametro `maxReceiveCount = 5` definisce che, al terzo tentativo fallito di elaborazione, SQS rimuove automaticamente il messaggio dalla coda principale e lo sposta nella DLQ. Il messaggio viene conservato nella DLQ per un periodo di *Message Retention* (es. 14 giorni), permettendo agli operatori di effettuare un'analisi *post-mortem* (*dead-letter analysis*) ed eseguire eventuale *re-drive* manuale dopo la correzione del bug.

3.3 Computazione Serverless e Modelli di Sicurezza IAM

3.3.1 AWS Lambda: Ciclo di Vita ed Ambiente di Esecuzione

AWS Lambda è un servizio di calcolo *event-driven* che esegue il codice in risposta a eventi senza richiedere il provisioning o la gestione di server.

Alla ricezione di un evento dalla coda SQS, l'infrastruttura di AWS crea un ambiente di esecuzione isolato (*microVM* basata su tecnologia **Firecracker**).

Il ciclo di vita della funzione si articola in tre fasi:

1. **Init Phase:** Download del pacchetto codice `.zip`, inizializzazione del runtime Python e caricamento dei moduli esterni o SDK (`boto3`).

   Durante questa fase è stata riscontrata un'anomalia critica legata al caricamento delle librerie esterne (es. `pypdf` e `google-genai`), sfociata in eccezioni di tipo `ModuleNotFoundError` e `ImportError` durante il cold start dell'ambiente di esecuzione Lambda.

   L'analisi del problema ha evidenziato una causa:

   - **Incompatibilità dell'Architettura Binaria (Cross-Compilation Mismatch):** L'esecuzione di un normale `pip install` gestito dall'ambiente di sviluppo locale (macOS) installa pacchetti ed eventuali librerie C/C++ ottimizzati per la piattaforma host, rendendoli del tutto incompatibili con l'architettura Linux target dell'ambiente AWS Lambda.

   La problematica è stata risolta con uno script di build basato sulle funzionalità di cross-installazione di `pip:`

   ```
   pip install \
   --platform manylinux2014_x86_64 \
   --target ./src/layers/python \
   --implementation cp \
   --python-version 3.11 \
   --only-binary=:all: \
   --no-cache-dir \
   -r src/layers/requirements.txt
   ```

2. **Invoke Phase:** Esecuzione del metodo handler (`main.lambda_handler`) passando l'evento contenente il record SQS.
3. **Shutdown Phase:** Disattivazione e pulizia dell'ambiente di esecuzione al termine dell'inattività.

Per la gestione delle librerie di terze parti (ad esempio parser PDF o client HTTP avanzati), la funzione sfrutta i **Lambda Layers**, risorse esterne collegate che permettono di mantenere snello il pacchetto di codice principale (`lambda.zip`) e riutilizzare dipendenze comuni.

3.3.2 Sicurezza IAM

Per garantire la sicurezza, la computazione serverless si basa sulla netta separazione tra **chi può assumere l'identità della Lambda** e **cosa tale identità è autorizzata a fare**.

Trust Policy

È una risorsa basata su politiche d'accesso che definisce quale entità può assumere il ruolo IAM definito. Per la Lambda, la Trust Policy autorizza esplicitamente il servizio gestito `lambda.amazonaws.com` tramite l'azione `sts:AssumeRole`.

```
# Define the IAM role for the Lambda function to assume when it is invoked. This role allo
ws the Lambda function to access AWS services and resources.
```

```
resource "aws_iam_role" "lambda_role" {
    name = "${local.name_prefix}-lambda-role"
    # Define the Trust Policy that allows the Lambda service to assume this role.
    assume_role_policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": "sts:AssumeRole"
            "Effect": "Allow",
            "Principal": {
                "Service": "lambda.amazonaws.com"
            },
        }
        ]
    })
}
```

Identity-Based Access Policy

È la politica di autorizzazione associata al ruolo IAM assunto dalla Lambda. In base al principio del minimo privilegio, essa definisce le sole azioni consentite sulle risorse target:

- **S3:** `s3:GetObject` e `s3:DeleteObject` sul bucket `cv-incoming-bucket`; `s3:PutObject` sul bucket `cv-archive-bucket`.
- **SQS:** `sqs:ReceiveMessage`, `sqs:DeleteMessage`, `sqs:GetQueueAttributes` sulla coda `cv-processing-queue`.
- **DynamoDB:** `dynamodb:PutItem` sulla tabella `CandidatesTable`.

```
# Define the IAM policy for the Lambda function to grant it the necessary permissions to a
ccess S3, SQS, DynamoDB, and CloudWatch Logs.
resource "aws_iam_role_policy" "lambda_policy" {
    name = "${local.name_prefix}-lambda-policy"
    role = aws_iam_role.lambda_role.id

    policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
        {
        "Sid": "S3IncomingBucketAccess",
        "Effect": "Allow",
        "Action": [
            "s3:GetObject",
            "s3:DeleteObject"
        ],
        "Resource": "${aws_s3_bucket.cv_incoming_bucket.arn}/*" # The Lambda function need
s to read and delete objects from the incoming S3 bucket.
        },
        {
        "Sid": "S3ArchiveBucketAccess",
        "Effect": "Allow",
        "Action": [
            "s3:PutObject"
        ],
        "Resource": "${aws_s3_bucket.cv_archive_bucket.arn}/*" # The Lambda function needs
to write objects to the archive S3 bucket.
        },
        {
        "Sid": "SQSServiceAccess",
        "Effect": "Allow",
```

```
        "Action": [
            "sqs:ReceiveMessage",
            "sqs:DeleteMessage",
            "sqs:GetQueueAttributes"
        ],
        "Resource": "${aws_sqs_queue.processing_queue.arn}" # The Lambda function needs to
receive and delete messages from the processing SQS queue.
        },
        {
        "Sid": "DynamoDBTableAccess",
        "Effect": "Allow",
        "Action": [
            "dynamodb:PutItem"
        ],
        "Resource": "${aws_dynamodb_table.candidates_table.arn}" # The Lambda function nee
ds to put items into the Candidates DynamoDB table.
        }
        ]
    })
}
```

3.4 Persistence Layer e Modellazione NoSQL

3.4.1 Amazon DynamoDB: Architettura Schemaless e Modalità Pay-Per-Request

Amazon DynamoDB è un database NoSQL di tipo chiave-valore e documento, completamente gestito, progettato per garantire latenze inferiori ai 10 millisecondi su qualsiasi scala.

Trattandosi di un database *schemaless*, non richiede la predefinizione della struttura delle colonne (eccetto la chiave primaria). La funzione Lambda può pertanto inserire liberamente oggetti JSON complessi e variabili generati dall'LLM.

La scelta della modalità `PAY_PER_REQUEST` **(On-Demand)** elimina la necessità di pianificare RCU (Read Capacity Units) e WCU (Write Capacity Units). La fatturazione avviene unicamente per le operazioni effettive di lettura e scrittura eseguite dalle funzioni Lambda o dai recruiter. (questa configurazione si presta ad essere adatta sia in scenari in cui il carico di lavoro è imprevedibile, o come in questo caso, scenari di test)

3.4.2 Modellazione degli Access Pattern e Global Secondary Index (GSI)

I database NoSQL richiedono una modellazione guidata dai pattern di accesso (*Access Pattern First*). Nel nostro sistema sono identificati due requisiti di lettura/scrittura:

1. **Scrittura e Ricerca per ID Candidato (Tabella Principale):** La Primary Key della tabella è composta da una sola Partition Key (Hash Key) denominata `CandidateID`. Consente l'inserimento (`PutItem`) ed il recupero puntuale di un singolo profilo in $O(1)$.
2. **Ricerca e Ranking per Ruolo Lavorativo (Recruiter Pattern):** Il recruiter necessita di estrarre la lista dei candidati idonei per una posizione (es. "Data Engineer") ordinata per punteggio di affinità. Poiché eseguire uno `Scan` dell'intera tabella principale sarebbe un'operazione lenta ed estremamente costosa, è stato definito un **Global Secondary Index (GSI)** chiamato `JobRoleScoreIndex`.

```
# Configuration for the DynamoDB table used to store candidate information.
resource "aws_dynamodb_table" "candidates_table" {
    name = local.dynamodb_table_name # Unique identifier for the DynamoDB table.
    billing_mode = "PAY_PER_REQUEST" # On-demand billing mode for the table.
    hash_key = "CandidateID" # Primary key for the table.

    attribute {
        name = "CandidateID" # Name of the primary key attribute.
        type = "S" # Type of the primary key attribute (String).
    }
```

```
    attribute {
        name = "job_id"
        type = "S"
    }

    attribute {
        name = "score"
        type = "N"
    }
    # Define a global secondary index (GSI) for querying candidates by job role and score.
    global_secondary_index {
        name = "JobRoleScoreIndex"
        key_schema {
            attribute_name = "job_id"
            key_type       = "HASH" # Hash key for the GSI.
        }
        key_schema {
            attribute_name = "score"
            key_type       = "RANGE" # Range key for the GSI.
        }
        projection_type = "ALL" # Copies all attributes from the table into the index.
  }
```

Nota Architetturale sulla Primary Key

Mantenendo `CandidateID` come *Hash Key* univoca della tabella principale, il sistema presuppone che un candidato possa avere **un solo CV attivo nel sistema**. Se lo stesso candidato si candida per due `job_id` diversi in tempi diversi, il record in DynamoDB verrà sovrascritto, aggiornando il suo score all'ultimo calcolato.

Struttura del GSI

job_id (Partition Key - HASH)	score (Sort Key - RANGE)	CandidateID (Primary Key Base)	full_name	email	matching_skills	archive_s3_uri
JOB-DEV-2026	**62**	`cand_103`	Giuseppe Verde	g.verde@email.com	`["Python"]`	`s3://archive/JOB-DEV-2026/g_verde.pdf`
JOB-DEV-2026	**85**	`cand_102`	Laura Bianchi	l.bianchi@email.com	`["AWS", "Python"]`	`s3://archive/JOB-DEV-2026/l_bianchi.pdf`
JOB-DEV-2026	**94**	`cand_101`	Mario Rossi	m.rossi@email.com	`["Terraform", "AWS", "Python"]`	`s3://archive/JOB-DEV-2026/m_rossi.pdf`
JOB-AI-2026	**78**	`cand_105`	Roberto Gialli	r.gialli@email.com	`["Python", "LLM"]`	`s3://archive/JOB-AI-2026/r_gialli.pdf`
JOB-AI-2026	**98**	`cand_104`	Anna Neri	a.neri@email.com	`["Gemini API", "PyTorch", "Python"]`	`s3://archive/JOB-AI-2026/a_neri.pdf`

3.5 Integrazione Event-Driven e Resource-Based Policies

S3 Bucket Notification

L'integrazione tra il bucket di ingressi e la coda SQS sfrutta il meccanismo nativo delle notifiche di evento S3. Quando la chiamata `s3:PutObject` del client si conclude con successo, Amazon S3 costruisce un payload JSON contenente i metadati dell'operazione (nome del bucket, chiave dell'oggetto, dimensione, ETag) e lo invia direttamente all'endpoint SQS.

SQS Queue Policy e Convalida della Sorgente

Poiché S3 è un servizio esterno alla coda SQS, quest'ultima necessita di una **Resource-Based Policy** (`aws_sqs_queue_policy`) per autorizzare il demone di notifica di S3 a scrivere messaggi al suo interno.

Per prevenire l'invio di messaggi non autorizzati da parte di bucket non appartenenti all'organizzazione, applichiamo una clausola restrittiva tale da consentire l'evento sulla coda SQS **esclusivamente** se l'ARN della risorsa mittente corrisponde in modo esatto all'ARN del bucket `cv-incoming-bucket`.

4. Implementazione dell'Infrastruttura

L'architettura del progetto adotta una suddivisione modulare sia per la parte di codice applicativo (funzioni serverless e dipendenze) sia per l'Infrastruttura as Code (IaC) gestita tramite Terraform.

4.1 Organizzazione dei Moduli

```
Progetto_dir/
│
├── src/
│   ├── lambda_api/
│   │   └── main.py
│   ├── lambda_parser/
│   │   └── main.py
│   ├── layers/
│   │   ├── python/
│   │   └── dependencies.zip
│   ├── lambda_api.zip
│   └── lambda_parser.zip
│
├── test_cv/
├── venv/
│
├── .gitignore
├── .terraform.lock.hcl
├── app.html
├── dynamodb.tf
├── gateway.tf
├── iam.tf
├── lambda.tf
├── main.tf
├── outputs.tf
├── readMe.md
├── requirements.txt
├── s3.tf
├── sqs.tf
├── terraform.tfvars
└── variables.tf
```

La cartella `src/` racchiude la logica applicativa isolata per microservizi Lambda e la gestione dei pacchetti binari:

- `src/lambda_api/`: Contiene il codice sorgente della funzione responsabile della gestione delle richieste HTTP provenienti dall'API Gateway (creazione posizioni, recupero candidati e generazione di URL pre-firmati).
- `src/lambda_parser/`: Ospita la logica di parsing asincrono (`main.py`) che si interfaccia con la libreria `pypdf` e con le API di Google Gemini per la valutazione dei CV.
- `src/layers/`: Raccoglie le dipendenze esterne Python compilate per l'ambiente Linux di esecuzione delle Lambda, impacchettate all'interno dell'archivio `dependencies.zip`.

I file di configurazione Terraform alla radice del progetto disaccoppiano le risorse cloud per competenza logica:

- `main.tf`: File principale di configurazione del provider AWS e dei blocchi di inizializzazione globali.
- `variables.tf` & `terraform.tfvars`: Definiscono i parametri configurabili dell'infrastruttura (es. nomi dei progetti, regioni AWS, chiavi API).
- `s3.tf`: Configura i bucket Amazon S3 dedicati all'ingestione (`cv-incoming-bucket`) e all'archiviazione a lungo termine (`cv-archive-bucket`).

```
# S3 bucket for incoming files (cv-incoming-bucket) configuration.
resource "aws_s3_bucket" "cv_incoming_bucket" {
    bucket = local.bucket_incoming_name # Identifier for the S3 bucket.
    force_destroy = true # Enable force_destroy to allow the bucket to be deleted even
```

```
if it contains objects. This is useful for testing environments.
    tags = {
        Name        = "CV Incoming Bucket"
        Environment = var.environment
    }
}

# Configuration of the S3 bucket notification to trigger the SQS queue when a new objec
t is created in the incoming bucket.
resource "aws_s3_bucket_notification" "incoming_bucket_notification" {
    bucket = aws_s3_bucket.cv_incoming_bucket.id

    queue {
        queue_arn     = aws_sqs_queue.processing_queue.arn
        events        = ["s3:ObjectCreated:*"]
    }
    # With depends_on attribute this resource will be created after the SQS queue polic
y is created.
    depends_on = [
        aws_sqs_queue_policy.processing_queue_policy
    ]
}

# S3 bucket for archive files (cv-archive-bucket) configuration. (DESTINATION BUCKET)
resource "aws_s3_bucket" "cv_archive_bucket" {
    bucket = local.bucket_archive_name
    force_destroy = true
    tags = {
        Name        = "CV Archive Bucket"
        Environment = var.environment
    }
}
```

- `dynamodb.tf`: Definisce le tabelle NoSQL DynamoDB (`jobs_table` e `candidates_table`), inclusa la configurazione del Global Secondary Index (`JobIdScoreIndex`).
- `sqs.tf`: Gestisce la coda di buffering degli eventi (`cv-processing-queue`), la Dead Letter Queue (`cv-dlq`) e la relativa *Redrive Policy*. Oltre ad abilitare i messaggi da parte della risorsa s3.
- `gateway.tf`: Configura l'Amazon API Gateway e le integrazioni proxy verso la funzioni Lambda dedicata.

```
resource "aws_apigatewayv2_api" "http_api" {
  name          = "${var.project-name}-http-api"
  protocol_type = "HTTP"

  cors_configuration {
    allow_origins = ["*"]
    allow_methods = ["GET", "POST", "OPTIONS"]
    allow_headers = ["Content-Type"]
    max_age       = 300
  }
}

# Define the integration between the API Gateway and the Lambda function.
resource "aws_apigatewayv2_integration" "lambda_integration" {
  api_id             = aws_apigatewayv2_api.http_api.id
  integration_type   = "AWS_PROXY"
  integration_method = "POST"
  integration_uri    = aws_lambda_function.api_backend.invoke_arn
```

```
}

# Define the routes for the API Gateway:
# 1. GET /jobs: This route retrieves job information from the Jobs DynamoDB table.
resource "aws_apigatewayv2_route" "get_jobs" {
  api_id    = aws_apigatewayv2_api.http_api.id
  route_key = "GET /jobs"
  target    = "integrations/${aws_apigatewayv2_integration.lambda_integration.id}"
}

# 2. POST /jobs: This route allows the creation of new job entries in the Jobs DynamoDB
table.
resource "aws_apigatewayv2_route" "post_jobs" {
  api_id    = aws_apigatewayv2_api.http_api.id
  route_key = "POST /jobs"
  target    = "integrations/${aws_apigatewayv2_integration.lambda_integration.id}"
}

# 3. DELETE /jobs/{job_id}: This route allows the deletion of a specific job entry from
the Jobs DynamoDB table based on the provided job_id.
resource "aws_apigatewayv2_route" "delete_job" {
  api_id    = aws_apigatewayv2_api.http_api.id
  route_key = "DELETE /jobs/{job_id}"
  target    = "integrations/${aws_apigatewayv2_integration.lambda_integration.id}"
}

# 4. GET /jobs/{job_id}/candidates: This route retrieves candidate information associat
ed with a specific job from the Candidates DynamoDB table based on the provided job_id.
resource "aws_apigatewayv2_route" "get_job_candidates" {
  api_id    = aws_apigatewayv2_api.http_api.id
  route_key = "GET /jobs/{job_id}/candidates"
  target    = "integrations/${aws_apigatewayv2_integration.lambda_integration.id}"
}

# 5. POST /upload-url: This route generates a pre-signed URL for uploading CVs to the I
ncoming S3 bucket.
resource "aws_apigatewayv2_route" "upload_route" {
  api_id    = aws_apigatewayv2_api.http_api.id
  route_key = "POST /upload"
  target    = "integrations/${aws_apigatewayv2_integration.lambda_integration.id}"
}
```

- `iam.tf`: Regola i ruoli di esecuzione e le policy di sicurezza basate sul principio del minimo privilegio per le funzioni Lambda.
- `lambda.tf` : Gestisce il deployment delle funzioni AWS Lambda e il loro collegamento agli eventi asincroni SQS e all'API Gateway.

```
# Define the event source mapping to connect the Lambda function to the SQS queue.
resource "aws_lambda_event_source_mapping" "cv_llm_parser_sqs_mapping" {
  event_source_arn = aws_sqs_queue.processing_queue.arn
  function_name    = aws_lambda_function.cv_llm_parser.arn
  batch_size       = 1

  enabled = true # Enable the event source mapping to allow the Lambda function to be t
riggered by messages in the SQS queue.
}
```

```
# Add permission to the API Lambda function to allow it to be invoked by the API Gatewa
y.
resource "aws_lambda_permission" "api_gw_permission" {
  statement_id  = "AllowExecutionFromAPIGateway"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.api_backend.function_name
  principal     = "apigateway.amazonaws.com"
  source_arn    = "${aws_apigatewayv2_api.http_api.execution_arn}/*/*"
}
```

Implementazioni Future

Monitoring e Alerting

Implementazione dell'allarme CloudWatch basato sulla metrica `ApproximateNumberOfMessagesVisible` della DLQ e inoltro delle notifiche via Amazon SNS verso gli operatori DevOps.